Contents

Monitoring & Observability — Interview Questions & Answers **1**

Monitoring & Observability — Interview Questions & Answers

Q1: What is the difference between monitoring and observability? > **Monitoring**: Watching pre-defined metrics and alerting when thresholds are breached. You know what to look for in advance. "Is the system up? Is latency above 200ms?" > **Observability**: The ability to understand internal system state from external outputs (metrics, logs, traces). You can explore unknown unknowns. A system is observable when you can ask arbitrary questions without deploying new code. > In practice: monitoring tells you *something is wrong*; observability tells you *why*.

Q2: Explain the difference between a Prometheus Counter, Gauge, and Histogram. When do you use each? > **Counter**: Monotonically increasing integer. Reset to 0 on restart. Always use `rate()` or `increase()` in queries — never query raw value. > Use for: requests_total, errors_total, bytes_sent_total > > **Gauge**: Can increase or decrease. Represents a snapshot. > Use for: memory_bytes, active_connections, queue_depth, temperature > > **Histogram**: Pre-defined buckets. Enables server-side percentile calculation across instances. `histogram_quantile(0.99, ...)` is accurate when aggregating across replicas. > Use for: request latency, response size — anything you need percentiles for > > Key distinction: use Histogram over Summary when you need to aggregate across multiple instances.

Q3: What is an error budget and how does it influence engineering decisions? > Error budget = 1 - SLO. If SLO is 99.9%, the error budget is 0.1% — about 43.8 minutes/month. > > How it drives behavior: > - **Budget remaining** → deploy freely, run experiments, take on tech debt > - **Budget 50% consumed** → be cautious, review risky deployments > - **Budget exhausted** → freeze all non-critical deployments until next period > > It creates a shared language between product (wants features fast) and engineering (wants reliability). Both sides agree on the acceptable risk level upfront.

Q4: Explain burn rate alerting. Why is it better than threshold-based alerting? > Burn rate = how fast you're consuming error budget relative to the allowed rate. > A 14.4x burn rate means you'll exhaust the entire monthly budget in 2 hours. > > **Why better than thresholds:** > - Threshold alerts: "error rate > 1%" — doesn't tell you the business impact > - Burn rate: "at this rate, you'll miss your SLO in 1 hour" — directly tied to user experience > - Reduces noise: short error spikes don't alert unless sustained > - Multi-window: fast burn (1h window) for critical pages + slow burn (6h window) for warning tickets

---

Q5: A key service is having intermittent latency spikes. How do you investigate? > 1. **Metrics first**: Check p99 latency graph over time. Is it all endpoints or specific ones? > 2. **Correlate with traffic**: Does spike correspond to traffic increase? (saturation) > 3. **Check dependencies**: Database query time, cache hit rate, external API latency > 4. **Traces**: Find slow traces during the spike window — identify which span is slow > 5. **Logs**: Filter logs for high `duration_ms` during spike — look for patterns > 6. **Infrastructure**: CPU throttling? GC pauses? Network retransmissions? > 7. **Deployments**: Any deploys correlate with the spike start? (check annotations on dashboard) > 8. **Node-level**: `kubectl top nodes`, check for noisy neighbor

---

Q6: What is the difference between push and pull metrics collection? > **Pull (Prometheus model)**: Scraper polls targets on a schedule. Centralized config. Easy to see which targets are down (scrape fails). Works well in dynamic environments with service discovery. > > **Push (StatsD, Datadog Agent, CloudWatch)**: Applications push metrics to a collector. Works well for short-lived jobs (Lambda, batch), firewalled environments, or when you can't open inbound ports. > > **Pushgateway** bridges the gap: batch jobs push to Pushgateway, Prometheus pulls from it.

Q7: How do you reduce alert fatigue? > 1. **Alert on symptoms not causes** — alert when user experience degrades, not every CPU spike > 2. **Burn rate instead of threshold** — sustained problems, not brief

spikes > 3. **Add `for` duration** — require problem sustained for 2-5 min before alerting > 4. **Inhibition rules** — suppress warning when critical is already firing > 5. **Proper grouping/routing** — database alerts go to DB team, not everyone > 6. **Review stale alerts** — delete or fix any alert that's been ignored 3+ times > 7. **Separate page vs ticket** — critical pages on-call, warnings create Jira tickets

---

Q8: Explain distributed tracing. What problem does it solve? > In a microservices architecture, a single user request touches 10+ services. When a request is slow or fails, traditional logs only show what happened inside each service — you can't see the full journey. > > Distributed tracing gives each request a unique `trace_id`. Every service adds a span with timing and metadata. You can visualize the full call graph and immediately see: > - Which service is slow (bottleneck) > - Which dependency call failed > - How much of the total latency is in each service > - How spans are nested (parallel vs sequential calls)

Q9: How does OpenTelemetry differ from Prometheus or Jaeger? > OpenTelemetry is a **vendor-neutral instrumentation framework** — it defines APIs and SDKs for generating metrics, logs, and traces. It doesn't store or visualize data. > > Prometheus and Jaeger are **backends** — they store and query data. > > Workflow: Instrument app with OTel SDK → OTel Collector receives data → forwards to Prometheus (metrics) + Jaeger/Tempo (traces) + Loki (logs). > > Benefit: Change backends without changing application code. Auto-instrumentation for HTTP, gRPC, databases, message queues.

Q10: Production scenario — Grafana shows error rate spiking to 30% but PagerDuty didn't alert. Why? > Possible reasons: > 1. Alert `for` duration too long (e.g., `for: 15m`) — spike resolved before threshold met > 2. Alert expression uses wrong labels — not matching this service's metric labels > 3. Prometheus scrape failing — metrics stale, alert never evaluates to true > 4. Alertmanager route misconfigured — alert routes to wrong receiver or suppressed > 5. Silence active in Alertmanager > 6. PagerDuty integration broken or routing key expired > > Debug steps: Check Prometheus `Alerts` tab, check Alertmanager `Status` page, verify `amtool alert query`, test fire alert manually.

Q11: What is the cardinality problem in Prometheus? > Cardinality = number of unique time series = unique combinations of label values. > High cardinality kills Prometheus performance because each series needs memory + storage. > > **Bad labels** (never do this): > - User IDs: `{user_id="u_12345"}` — millions of series > - Request IDs, URLs with query params > - Timestamps in labels > > **Good labels**: service, method, status_code, region, environment > > Rule: if a label has > 100 unique values, think carefully. If > 1000, almost always wrong. > High-cardinality data belongs in logs or traces, not metrics.

Q12: What logging strategy do you use for a Kubernetes cluster with 50+ services? > Architecture: > - **Fluent Bit** as DaemonSet on every node — lightweight, low CPU/memory, reads container logs from `/var/log/containers/` > - Enriches with K8s metadata (pod, namespace, deployment, labels) > - Forwards to **Loki** (cost-efficient) or **OpenSearch** (rich full-text search) > - **Grafana** for log exploration + correlation with metrics > > Standards enforced: > - All services output structured JSON logs > - Mandatory fields: `timestamp`, `level`, `service`, `trace_id`, `span_id` > - Log correlation: inject `trace_id` in all logs — click trace in Grafana → jump to Jaeger > - Log retention policy: 30 days hot, archive to S3 Glacier